DevSecOps Lab - Study Guide

A practical guide to this Python CI/CD learning project

1. What is this project?

This is a small learning project that demonstrates Continuous Integration (CI) and a basic Continuous Delivery (CD) packaging step. The calculator app is intentionally simple so you can focus on how automated checks work whenever code is pushed or a pull request is opened.

The workflow runs a linter and unit tests first. Only if those checks pass does a second job build a Docker image. GitHub Actions records each run, its result, and its duration.

2. What each file does

- app.py - defines add(a, b) and divide(a, b). Division by zero raises ValueError.
- test_app.py - contains three pytest checks: addition, normal division, and division by zero.
- requirements.txt - installs pytest for tests and flake8 for linting.
- Dockerfile - packages the project in a Python 3.11 slim container. Its default command runs pytest.
- .github/workflows/python-ci.yml - tells GitHub Actions when and how to run lint, tests, and the image build.
- .gitignore - excludes Python cache files and virtual environments from Git.

3. Pipeline at a glance

```
Push or pull request
        |
        v
build-test job: install -> flake8 -> pytest
        | only if successful
        v
docker-build job: docker build
        |
        v
GitHub Actions records status and duration
```

A failed lint or test stops the dependent Docker job. This avoids spending time packaging code that has already failed a quality check.

4. How the workflow works

The workflow is stored at .github/workflows/python-ci.yml. GitHub recognizes YAML files in this folder and runs the workflow for repository events.

Triggers and permissions

```
on:
  push:
  pull_request:

permissions:
  contents: read
```

push runs after commits are pushed. pull_request runs when a pull request is opened or updated. contents: read gives the workflow a read-only repository token; it does not need permission to change source code.

The build-test job

- Runs on a fresh Ubuntu runner hosted by GitHub.
- Checks out the repository, sets up Python 3.11, and installs requirements.txt.
- Runs python -m flake8 app.py test_app.py to check style and common errors.
- Runs python -m pytest -q to execute the unit tests.

The docker-build job

```
docker-build:
  needs: build-test
  steps:
    - uses: actions/checkout@v4
    - run: docker build --tag devsecops-lab:${{ github.sha }} .
```

needs: build-test enforces job order. The Docker job starts only after build-test succeeds. The image is built on the runner, tagged with the commit SHA, and is not pushed to a container registry.

5. What Docker does here

A Docker image is a packaged filesystem and instructions for running software. The Dockerfile starts from python:3.11-slim, sets /app as the working directory, installs the requirements, copies in the project, and sets pytest as the default command.

```
docker build -t devsecops-lab .
docker run --rm devsecops-lab
```

The first command builds the image; the second runs its default test command and then removes the container. Docker must be installed locally to run these commands. GitHub Actions can build it on its hosted runner.

6. Run the checks locally

```
python -m pip install -r requirements.txt
python -m flake8 app.py test_app.py
python -m pytest -q
```

Expected test result: 3 passed. If lint or tests fail, fix the code, rerun the checks, then commit and push. A new push starts another workflow run.

7. Current scope and what comes later

- This is a demonstration, not a deployed web service. The Python functions are not exposed through a web API.
- The Docker image is built but not published to Docker Hub or another registry.
- The Dockerfile defaults to running tests, so it is currently a test image rather than a production app image.
- Security scanning is a future extension. Typical additions include dependency vulnerability checks, Bandit for Python security findings, and container image scanning.
- A later deployment stage would need a real runtime command, a registry, deployment target, and carefully scoped credentials.

8. Key terms

- CI (Continuous Integration): automatically check code changes frequently.
- CD (Continuous Delivery/Deployment): automatically prepare or release tested software.
- Linting: static checks for style and common code problems; it does not prove the program is correct.
- Unit test: a small automated check of one part of a program.
- Container image: a packaged template used to create containers.
- Workflow/job/step: a GitHub Actions workflow contains jobs; each job contains steps.
- Dependency: an external package the project installs, such as pytest.

9. Quick review

Try answering these without looking back:

- What must pass before the Docker build job starts?
- What does contents: read permit the workflow to do?
- Does this workflow publish the built image?
- What happens when divide(1, 0) is called?

Answers: lint and tests; read repository contents only; no, it only builds; the function raises ValueError.